The Journal of Open Source Software

Meteora: unified access and processing of meteorological station data in Python

Martí Bosch [1,2]

1 Center for Climate Systems Modeling C2SM, ETH Zurich, Zürich, Switzerland **2** Federal Office of Meteorology and Climatology MeteoSwiss, Zürich, Switzerland

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 11 July 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Ground-based meteorological station observations are central to research across a wide variety of fields including atmospheric science, hydrology, agriculture and urban climate. Nevertheless, working with these observations in practice can be laborious: data providers, e.g., national weather services, research and agricultural networks, local or regional low-cost sensor (LCS) networks, and crowdsourced citizen weather stations (CWS), usually expose different API or file formats, station-metadata schemas, variable naming, unit conventions and time references. Researchers therefore spend substantial effort writing code to acquire and preprocess data before any analysis can begin.

In order to streamline this process, Meteora defines a **common data model** for in-situ station observations and provides an extensible set of provider clients that map heterogeneous sources onto this unified representation. Each provider is implemented as a `BaseClient` subclass exposing the same interface, so that for every supported provider:

- the `stations_gdf` property provides the station metadata as a geopandas geo-data frame (Fleischmann et al., 2026) with point geometries that encode station locations in an explicit coordinate reference system
- the `get_ts_df` method provides the measurements as a timezone-aware pandas time series data frame (McKinney, 2010), whose variables follow a controlled vocabulary based on the surface meteorological Essential Climate Variables (ECVs) defined by the Global Climate Observing System (Bojinski et al., 2014) and carry explicit physical units through Pint (Grecco & Pint contributors, 2024).

These measurements can be obtained in the long- and wide-form data-frame layouts common to statistical and time-series workflows (Wickham, 2014), and can additionally be assembled into vector data cubes (Pebesma, 2022) using the xvec package (Fleischmann & xvec contributors, 2024), providing a single data structure that indexes measurements simultaneously in space and time. Because every provider shares this common interface, the same `get_ts_df` call returns the same, analysis-ready structure regardless of the underlying provider, so downstream code can be written once and run seamlessly against any source.

This harmonized data model is the foundation on which Meteora builds **provider-agnostic capabilities**:

- a configurable **quality control (QC) pipeline** of composable *flagging* and *transforming* steps that can be reordered, parameterized or extended with user-defined callables, based on the CWS QC methods of Napoly et al. (2018) and CrowdQC+ (Fenner et al., 2021) to detect mislocated, unreliable, indoor and radiatively-biased stations;
- **radiation-bias correction** (Bosch & Burger, 2026) of air temperature measurements, designed for LCS, with corrections specific to each sensor model; and

DRAFT

- the computation of **climate indices** over station observations with xclim (Bourgault et al., 2023) and xarray (Hoyer & Hamman, 2017), tools conventionally applied to gridded climate data.

The result is a single, coherent interface that makes it easy to acquire, harmonize and process station observations from any provider, turning heterogeneous sources into analysis-ready data.

Statement of need

For surface meteorological data, globally harmonized sources can broadly be categorized into two main classes, i.e., gridded products such as the ERA5 reanalysis and global station compilations such as the Global Historical Climatology Network (GHCN). While reanalysis products directly integrate station observations using data assimilation pipelines, they tend to operate at spatial scales too coarse for many applications as well as to smooth away local extremes. Global station compilations, in turn, ingest only the networks exchanged internationally (largely official synoptic and airport/aviation stations, often the same ones the reanalyses assimilate), leaving them too sparse for data-scarce regions (Giesen et al., 2014) and for resolving urban climate (Baklanov et al., 2018; Muller et al., 2013). Capturing this fine-scale variability requires combining the national, regional and crowdsourced networks those products omit (Figure 1) — the central motivation behind Meteora.

DRAFT

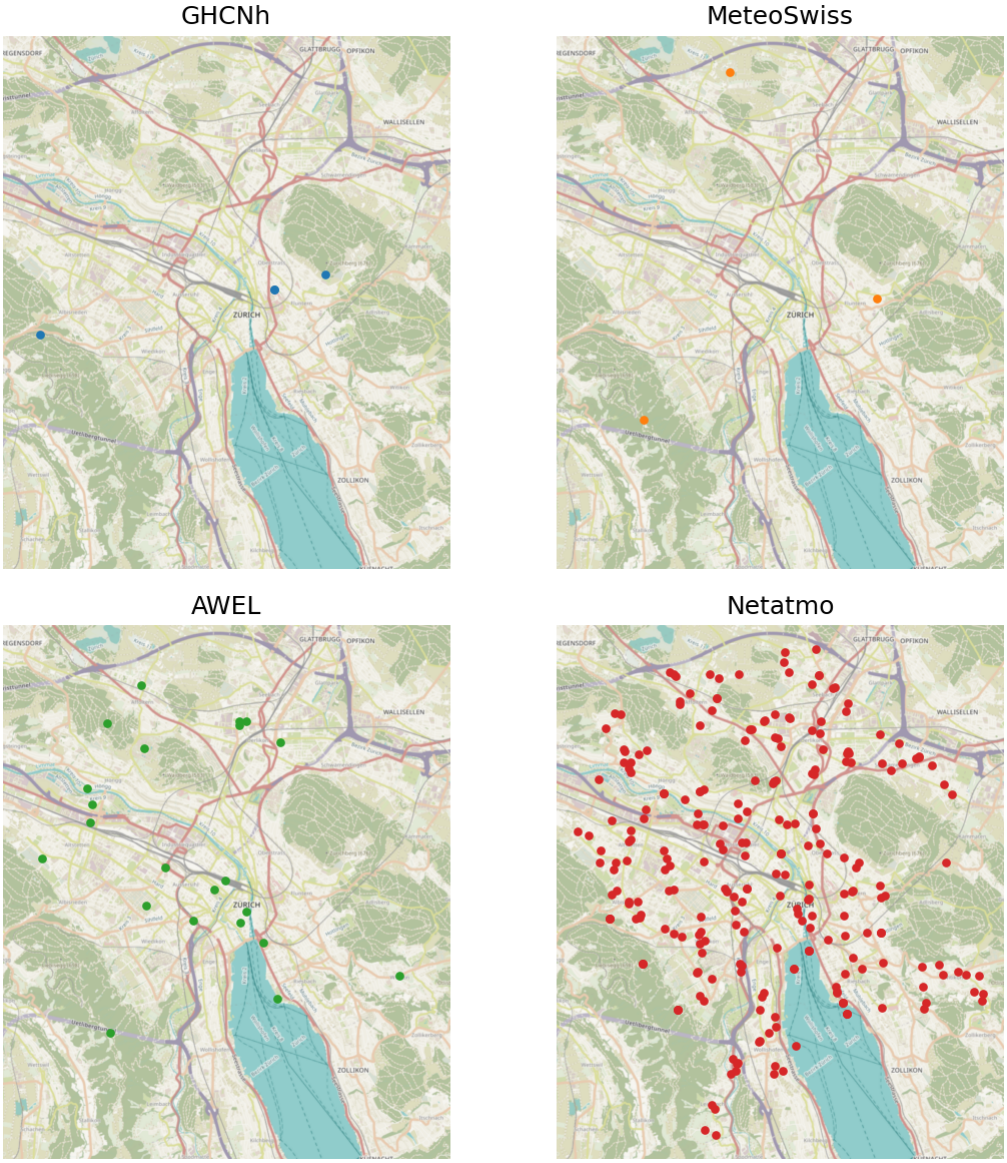


Figure 1: Station coverage for the Zürich (Switzerland) area from four networks accessible through Meteora: the global GHCN-hourly compilation, the national MeteoSwiss network, the cantonal AWEL network, and the Netatmo citizen weather stations.

Meteora targets researchers and practitioners who need to access station observations from potentially many heterogeneous sources, harmonize them into a single analysis-ready dataset, and apply common processing and analysis such as quality control, radiation-bias correction or climate-index computation, without writing ad-hoc per-provider code. Example use cases include urban climate modeling, station-density and network-representativeness assessments, comparison of CWS against reference observations, or evaluation of weather-forecast models against in-situ observations, e.g., with stationbench (Jua.ai, 2025; Molinaro et al., 2025).

State of the field

Accessing meteorological station data is fragmented across providers. In some cases, individual networks are served by single-source packages that return data in a provider-specific schema and

are often unmaintained. An example of a broader tool is meteostat (Lamprecht, 2021), which aggregates a curated set of stations into its own store and lets users openly access a harmonized collection through its API and Python package. However, meteostat covers mainly official networks — omitting the regional and crowdsourced stations that densify local, and especially urban, observations — and exposes no abstraction for users to add providers of their own. On the other hand, wetterdienst (Gutzmann & Motl, 2026) fetches and harmonizes directly from several national weather services under one interface, making it the closest analog to Meteora; nevertheless it shares the same limitations as meteostat, i.e., supporting a fixed, built-in provider set covering only official networks. Commercial APIs such as Synoptic (exposed in Python through the SynopticPy package) require a paid subscription and remain provider-mediated. There is consequently no unified, extensible layer that covers official, regional, low-cost and crowdsourced networks, lets users easily add clients for new ones, and turns their heterogeneous observations into a single, standardized, analysis-ready representation.

This data harmonization transforms observations into a single, analysis-ready format so that the downstream tasks that normally demand ad-hoc preprocessing become straightforward. A notable example is quality control, which is usually done through dedicated packages such as TITAN (B et al., 2020), CrowdQC+ (Fenner et al., 2021) and the MetObs-toolkit (Vergauwen et al., 2024), which assume the user already holds harmonized files — Meteora instead couples acquisition and quality control within one interface. Similarly, climate indices can be computed directly on this harmonized output with xclim (Bourgault et al., 2023), thus eliminating the need of per-provider preprocessing that such analyses would otherwise require.

Software design

A central goal of Meteora is to make supporting a new weather service straightforward. Each provider is a `BaseClient` subclass built on a small collection of endpoint-specific **abstract base classes** (for JSON, plain-text and file-based endpoints) together with composable mixins that encapsulate the recurring patterns of station-data retrieval: partitioning requests by station, time period or variable as required by each API, request caching, and the harmonization steps of variable renaming, unit attachment and timezone localization. Because every provider is a thin mapping onto the shared data model, adding a source amounts to writing a new `BaseClient` subclass: declaring a handful of attributes (endpoints, identifier columns, the ECV mapping and the source timezone) and implementing a parsing method that reshapes the provider's response into the common long form. The new client then exposes the same harmonized `stations_gdf` and `get_ts_df` interface. A step-by-step guide to writing a client is included in the documentation. The released version ships clients spanning official, regional and crowdsourced networks: AEMET (Spain), Agrometeo and AWEL (Switzerland), the Iowa Environmental Mesonet (ASOS and METAR networks), NOAA GHCN-hourly, Meteocat (Catalonia), MeteoSwiss, and the Netatmo citizen weather station network.

Research impact statement

Meteora has been used to produce the observational basis of two studies of urban climate in Switzerland:

- Bosch & Burger (2026) used Meteora in three stages. First, the bias-correction module was used to train sensor-model-specific radiation-bias corrections for several models of LCS, co-located alongside a MeteoSwiss station during summer 2025. Second, in each of the studied cities, Meteora was used to acquire data from the MeteoSwiss stations and, where available, from the local LCS network (e.g., AWEL in Zürich). Third, the climate-indices module was used to compare urban heat indices computed from reference (MeteoSwiss) stations only, from raw LCS and from radiation-corrected LCS, showing that integrating corrected LCS into the official network can substantially change the results.

DRAFT

The full reproducible workflow is publicly available at github.com/martibosch/swiss-uhi-lcd.

- Bosch et al. (2026) used Meteora to acquire and harmonize observations from the MeteoSwiss and AWEL networks into a single dataset, and to detect heatwave periods from the MeteoSwiss observations. The full reproducible workflow is publicly available at github.com/martibosch/zurich-uhi-lur.

Both radiation-bias correction and heatwave detection are illustrated with executable example notebooks in the Meteora user guide.

Availability

Meteora is released under the GPL-3.0-or-later license. The source code has been publicly developed on GitHub at github.com/martibosch/meteora since 2023, and versioned releases are distributed via the Python Package Index and conda-forge since October 2024. The documentation is hosted on Read the Docs and includes the API reference, a user guide with executable example notebooks that showcase the capabilities and applications of Meteora as well as developer guidelines for contributions. Continuous integration runs both the pre-commit style checks using pre-commit.ci and the unit-test suite on every push via GitHub Actions, with test coverage reported on Codecov.

AI usage disclosure

Generative AI assistance (Anthropic Claude, models in the Claude 4 family) was used during development for code refactoring, drafting documentation, and copy-editing this manuscript. All AI-assisted output was reviewed and validated by the author, who made all design and architectural decisions.

Acknowledgements

Meteora builds on the scientific Python ecosystem, in particular pandas (McKinney, 2010), NumPy (Harris et al., 2020), geopandas (Fleischmann et al., 2026), xarray (Hoyer & Hamman, 2017), xclim (Bourgault et al., 2023), Pint (Grecco & Pint contributors, 2024) and Pooch (Uieda et al., 2020).

References

B, L., Lussana, C., Nipen, T. N., Seierstad, I. A., Oram, L., & Aspelien, T. (2020). TITAN automatic spatial quality control of meteorological in-situ observations. *Advances in Science and Research*, *17*, 153–163.

Baklanov, A., Grimmond, C. S. B., Carlson, D., Terblanche, D., Tang, X., Bouchet, V., & Hovsepyan, A. (2018). From urban meteorology, climate and environment research to integrated city services. *Urban Climate*, *23*, 330–341. https://doi.org/10.1016/j.uclim.2017.05.004

Bojinski, S., Verstraete, M., Peterson, T. C., Richter, C., Simmons, A., & Zemp, M. (2014). The concept of essential climate variables in support of climate research, applications, and policy. *Bulletin of the American Meteorological Society*, *95*(9), 1431–1443. https://doi.org/10.1175/BAMS-D-13-00047.1

Bosch, M., Brandi, A., Bachofen, C., Hamel, P., Remme, R. P., Ramsay, E. E., Manoli, G., Schlaepfer, M. A., & Burger, M. (2026). *Urban climate regressions require representative monitoring networks*. https://arxiv.org/abs/2610.29423

DRAFT

Bosch, M., & Burger, M. (2026). *Revisiting urban heat indices in switzerland using low-cost measurement networks*. https://arxiv.org/abs/2606.09364

Bourgault, P., Huard, D., Smith, T. J., & others. (2023). Xclim: Xarray-based climate data analytics. *Journal of Open Source Software*, *8*(85), 5415. https://doi.org/10.21105/joss.05415

Fenner, D., Bechtel, B., Demuzere, M., Kittner, J., & Meier, F. (2021). CrowdQC+—a quality-control for crowdsourced air-temperature observations enabling world-wide urban climate applications. *Frontiers in Environmental Science*, *9*, 720747. https://doi.org/10.3389/fenvs.2021.720747

Fleischmann, M., Van den Bossche, J., Jordahl, K., Richards, M. J., McBride, J., Wasserman, J., Ward, B., & Wolf, L. J. (2026). GeoPandas: Fundamental data structures for vector spatial data in python. *Computers, Environment and Urban Systems*, *130*, 102495. https://doi.org/10.3389/fenvs.2021.720747

Fleischmann, M., & xvec contributors. (2024). *Xvec: Vector data cubes for Xarray*. https://github.com/xarray-contrib/xvec.

Giesen, N. van de, Hut, R., & Selker, J. (2014). The trans-african hydro-meteorological observatory (TAHMO). *Wiley Interdisciplinary Reviews: Water*, *1*(4), 341–348. https://doi.org/10.1002/wat2.1034

Grecco, H. E., & Pint contributors. (2024). *Pint: Makes units easy*. https://github.com/hgrecco/pint.

Gutzmann, B., & Motl, A. (2026). *Wetterdienst: Open weather data for humans*. Zenodo. https://doi.org/10.5281/zenodo.21048785

Harris, C. R., Millman, K. J., Walt, S. J. van der, & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, *5*(1), 10. https://doi.org/10.5334/jors.148

Jua.ai. (2025). *Stationbench: Benchmarking weather forecasts against ground-station observations*. GitHub. https://github.com/juaAI/stationbench

Lamprecht, C. (2021). *Meteostat: Access to open weather and climate data*. https://meteostat.net.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

Molinaro, R., Siegenheim, N., Poulsen, N., Daubinet, J. D., Martin, H., Frey, M., Thiart, K., & Gabler, M. V. (2025). *EPT-2 technical report*. https://arxiv.org/abs/2507.09703

Muller, C., Chapman, L., Grimmond, C. S. B., Young, D., & Cai, X. (2013). Sensors and the city: A review of urban meteorological networks. *International Journal of Climatology*, *33*(7), 1585–1600. https://doi.org/10.1002/joc.3678

Napoly, A., Grassmann, T., Meier, F., & Fenner, D. (2018). Development and application of a statistically-based quality control for crowdsourced air temperature data. *Frontiers in Earth Science*, *6*, 118. https://doi.org/10.3389/feart.2018.00118

Pebesma, E. (2022). *Vector data cubes*. r-spatial.org blog. https://r-spatial.org/r/2022/09/12/vdc.html

Uieda, L., Soler, S. R., Rampin, R., & others. (2020). Pooch: A friend to fetch your data files. *Journal of Open Source Software*, *5*(45), 1943. https://doi.org/10.21105/joss.01943

Vergauwen, T., Vieijra, M., Covaci, A., Jacobs, A., Top, S., Dewettinck, W., Vandelanotte, K., Hellebosch, I., & Caluwaerts, S. (2024). MetObs - a Python toolkit for using non-

DRAFT

traditional meteorological observations. *Journal of Open Source Software*, *9*(95), 5916. https://doi.org/10.21105/joss.05916

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, *59*, 1–23.

DRAFT